

1. Creamos una carpeta del proyecto la abrimos con code .

```
Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

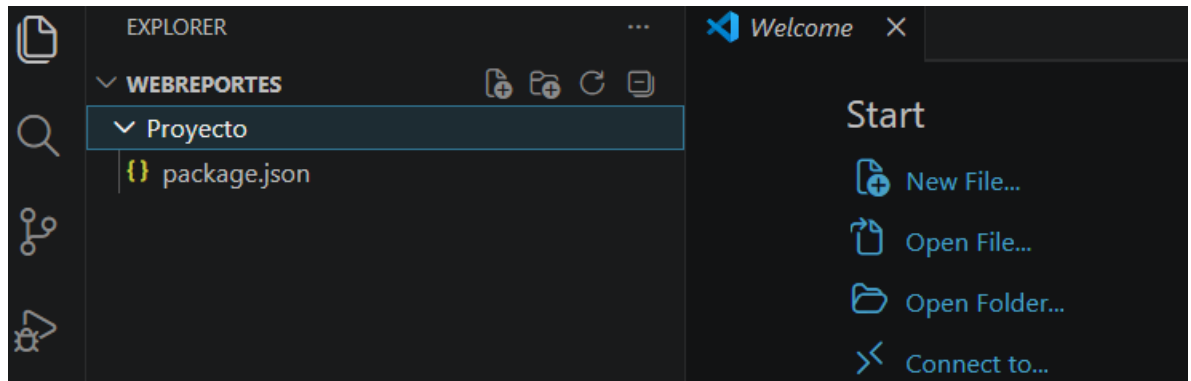
C:\Users\fmher>cd Webreportes
El sistema no puede encontrar la ruta especificada.

C:\Users\fmher>cd C:\Users\fmher\OneDrive\Documentos\WebReportes

C:\Users\fmher\OneDrive\Documentos\WebReportes> code .

C:\Users\fmher\OneDrive\Documentos\WebReportes>|
```

2. Luego de estar dentro del proyecto agrague una carpeta par mi caso Proyecto. Ubíquese en la carpeta y cree la paquete JS con el comando npm init -y , debe quedar así como muestra la imagen



3. Instale la siguiente librería que utilizara para los reportes en excell

```
  "author" : ,
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto> npm install exceljs
```

4. Debe salirle algo así :

```

PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto> npm install exceljs^C
PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto> npm install exceljs
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it.
  if you want a good and tested way to coalesce async requests by a key value, which is much more c
erful.
npm warn deprecated lodash.isequal@4.5.0: This package is deprecated. Use require('node:util').isD
ad.
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publici
bilities, which have been fixed in the current version. Please update. Support for old versions ma
xorbitant rates) by contacting i@izs.me
npm warn deprecated fstream@1.0.12: This package is no longer supported.
npm warn deprecated uuid@8.3.2: uuid@10 and below is no longer supported. For ESM codebases, upda
For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).

added 97 packages, and audited 98 packages in 7s

6 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto>

```

5. Altere la tabla en su base de datos, agregue el campo archivos

- ALTER TABLE** usuarios

ADD archivo **VARCHAR(255);**

id	nombre	email	id_rol
11	juan	jaun@gmail.com	NULL
13	diego lopez	diego@gmail.com	NULL
15	Alejandro	alejandro@gmail.com	NULL
16	Flor Hernandez P	fmhernandez@unicauca.edu.co	NULL
19	hugo	jjjggg	NULL

6. Consulte la tabla para comprobar la alteración

23

24 • `select * from usuarios`

25

	id	nombre	email	id_rol	archivo
▶	11	juan	jaun@gmail.com	NULL	NULL
	13	diego lopez	diego@gmail.com	NULL	NULL
	15	Alejandro	alejandro@gmail.com	NULL	NULL
	16	Flor Hernandez P	fmhernandez@unicauca.edu.co	NULL	NULL
	19	hugo	jjjggg	NULL	NULL

7. Vamos a utilizar las siguientes librerías para que tenga en cuenta
 - **multer** → sirve para subir archivos
 - **exceljs** → sirve para exportar Excel
 - **pdfkit** → sirve para generar reportes en PDF
8. En el proyecto instalar lo siguiente , son las dependencias mencionadas anteriormente.

```

found 0 vulnerabilities
PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto> ^C
PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto> npm install express mysql2 multer exceljs pdfkit cors morgan
an
:

```

9. Debe quedar así : ahí ya tenemos todas las dependencias que requiere

```

added 111 packages, and audited 209 packages in 6s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\fmher\OneDrive\Documentos\WebReportes\Proyecto>

```

10. Cree la siguiente arquitectura para el proyecto , agregue los archivos

En Proyecto agregue los archivos :

- server.js
- database.js
- uploads.js

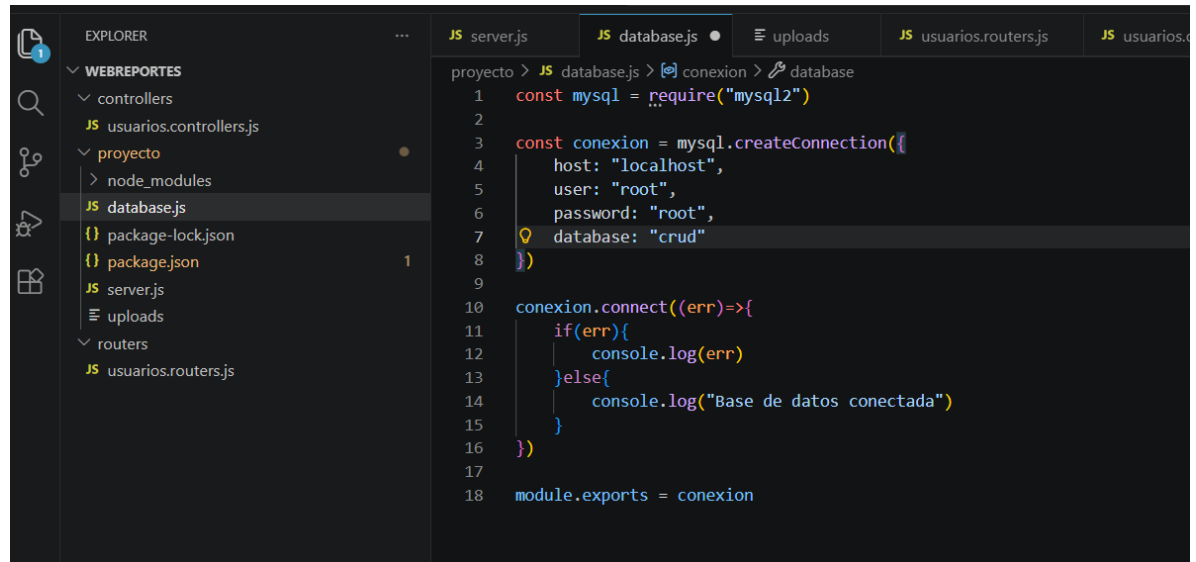
En routes

➤ usuarios.routes.js

Controllers

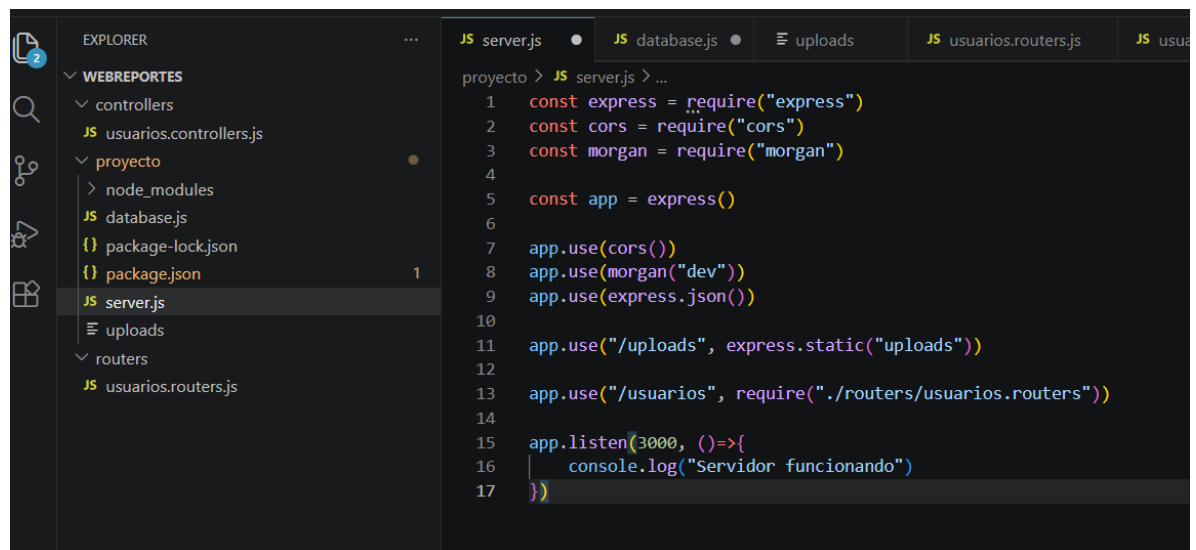
➤ usuarios.controllers.js

11. Creamos el archivo de conexión agregue el siguiente código en database.js



```
proyecto > JS database.js > conexion > database
1  const mysql = require("mysql2")
2
3  const conexion = mysql.createConnection({
4    host: "localhost",
5    user: "root",
6    password: "root",
7    database: "crud"
8  })
9
10 conexion.connect((err)=>{
11   if(err){
12     console.log(err)
13   }else{
14     console.log("Base de datos conectada")
15   }
16 })
17
18 module.exports = conexion
```

12. En el archivo server.js agregue lo siguiente :



```
proyecto > JS server.js > ...
1  const express = require("express")
2  const cors = require("cors")
3  const morgan = require("morgan")
4
5  const app = express()
6
7  app.use(cors())
8  app.use(morgan("dev"))
9  app.use(express.json())
10
11 app.use("/uploads", express.static("uploads"))
12
13 app.use("/usuarios", require("../routers/usuarios.routes"))
14
15 app.listen(3000, ()=>{
16   console.log("Servidor funcionando")
17 })
```

13. Al archivo routers agregue lo siguiente:

The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure: WEBREPORTES, controllers, usuarios.controllers.js, proyecto, node_modules, database.js, package-lock.json, package.json, server.js, uploads, routers, and usuarios.routers.js. The main editor area shows the code for usuarios.routers.js, which imports express and the usuarios.controllers module, and defines routes for creating, listing, updating, and deleting users, as well as generating Excel and PDF reports.

```
1 const router = require("express").Router()
2
3 const controller = require("../controllers/usuarios.controllers")
4
5 router.post("/", controller.crearUsuario)
6
7 router.get("/", controller.listarUsuarios)
8
9 router.put("/:id", controller.actualizarUsuario)
10
11 router.delete("/:id", controller.eliminarUsuario)
12
13 router.get("/excel/reporte", controller.reporteExcel)
14
15 router.get("/pdf/reporte", controller.reportePDF)
16
17 module.exports = router
```

14. En controllers/usuarios.controller.js agregue :

The screenshot shows the VS Code interface with the file explorer on the left and the code for usuarios.controllers.js in the main editor. The file explorer shows the same project structure as the previous screenshot, with usuarios.controllers.js selected. The main editor shows the code for usuarios.controllers.js, which imports database, multer, ExcelJS, and pdfkit. It configures multer for disk storage and defines the crearUsuario function, which handles file uploads and inserts user data into the database.

```
1 const db = require("../database")
2 const multer = require("multer")
3 const ExcelJS = require("exceljs")
4 const PDFDocument = require("pdfkit")
5
6 // CONFIGURAR MULTER
7 const storage = multer.diskStorage({
8
9   destination: (req, file, cb) => {
10     cb(null, "uploads/")
11   },
12
13   filename: (req, file, cb) => {
14     cb(null, Date.now() + "-" + file.originalname)
15   }
16 })
17
18 const upload = multer({storage}).single("archivo")
19
20 // CREAR
21 exports.crearUsuario = (req, res) => {
22
23   upload(req, res, (err) => {
24
25     if(err) return res.send(err)
26
27     const {nombre, email} = req.body
28
29     const archivo = req.file ? req.file.filename : null
30
31     db.query(
32       "INSERT INTO usuarios(nombre,email,archivo) VALUES(?,?,?)",
33       [nombre, email, archivo],
34       (err, result) => {
35         if(err) return res.send(err)
36
37         res.json({
```

```
21 exports.crearUsuario = (req,res)=>{
22   upload(req,res,(err)=>{
23     (err,result)=>{
24       if(err) return res.send(err)
25       res.json({
26         mensaje:"Usuario creado"
27       })
28     }
29   })
30 }
31
32 // LISTAR
33 exports.listarUsuarios = (req,res)=>{
34   db.query(
35     "SELECT * FROM usuarios",
36     (err,datos)=>{
37       if(err) return res.send(err)
38       res.json(datos)
39     }
40   )
41 }
42
43 // ACTUALIZAR
44 exports.actualizarUsuario = (req,res)=>{
45   const {id} = req.params
46   const {nombre,email} = req.body
```

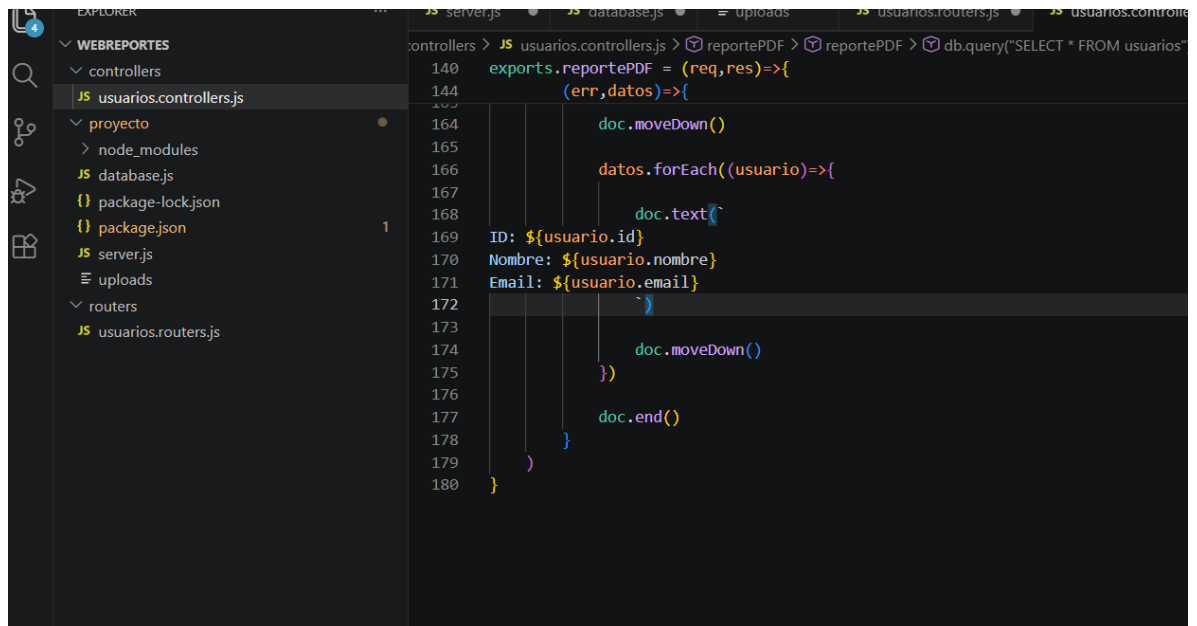
```
60 exports.actualizarUsuario = (req,res)=>{
61   [nombre,email,id],
62   (err,result)=>{
63     if(err) return res.send(err)
64     res.json({
65       mensaje:"Usuario actualizado"
66     })
67   }
68 }
69
70 // ELIMINAR
71 exports.eliminarUsuario = (req,res)=>{
72   const {id} = req.params
73   db.query(
74     "DELETE FROM usuarios WHERE id=?",
75     [id],
76     (err,result)=>{
77       if(err) return res.send(err)
78       res.json({
79         mensaje:"Usuario eliminado"
80       })
81     }
82   )
83 }
84
85 // REPORTE EXCEL
86 exports.reporteExcel = async (req,res)=>{
```

```
WEBREPORTES
├── controllers
│   └── usuarios.controllers.js
├── proyecto
│   ├── node_modules
│   ├── database.js
│   ├── package-lock.json
│   ├── package.json
│   ├── server.js
│   ├── uploads
│   └── routers
│       └── usuarios.routers.js
└── OUTLINE

controllers > JS usuarios.controllers.js > reportePDF > reportePDF
100 exports.reporteExcel = async (req,res)=>{
104   async (err,datos)=>{
107   }
108   const workbook = new ExcelJS.Workbook()
109   const worksheet = workbook.addWorksheet("Usuarios")
110   worksheet.columns = [
111     {header:"ID", key:"id", width:10},
112     {header:"Nombre", key:"nombre", width:30},
113     {header:"Email", key:"email", width:30}
114   ]
115   datos.forEach((usuario)=>{
116     worksheet.addRow(usuario)
117   })
118   res.setHeader(
119     "Content-Type",
120     "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
121   )
122   res.setHeader(
123     "Content-Disposition",
124     "attachment; filename=usuarios.xlsx"
125   )
126   await workbook.xlsx.write(res)
127   res.end()
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
```

```
WEBREPORTES
├── controllers
│   └── usuarios.controllers.js
├── proyecto
│   ├── node_modules
│   ├── database.js
│   ├── package-lock.json
│   ├── package.json
│   ├── server.js
│   ├── uploads
│   └── routers
│       └── usuarios.routers.js
└── OUTLINE

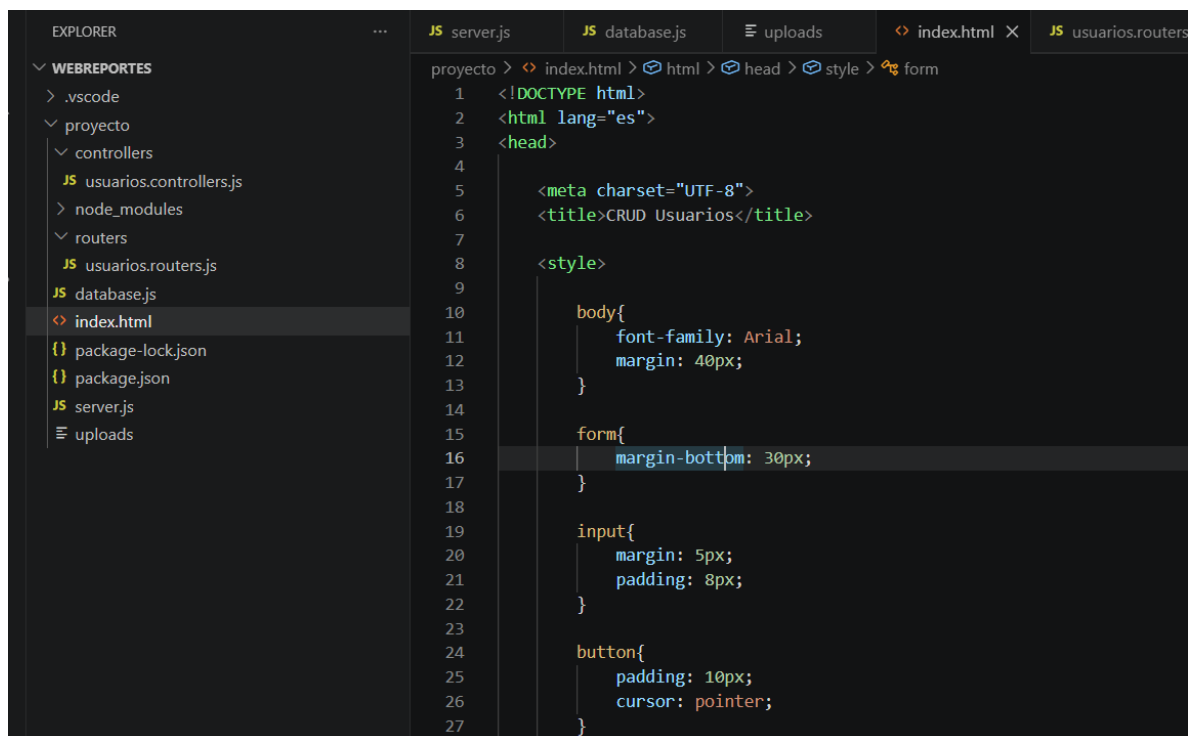
controllers > JS usuarios.controllers.js > reportePDF > reportePDF
100 exports.reporteExcel = async (req,res)=>{
136   }
137 }
138 // REPORTE PDF
139 exports.reportePDF = (req,res)=>{
140   db.query(
141     "SELECT * FROM usuarios",
142     (err,datos)=>{
143       if(err) return res.send(err)
144       const doc = new PDFDocument()
145       res.setHeader(
146         "Content-Type",
147         "application/pdf"
148       )
149       res.setHeader(
150         "Content-Disposition",
151         "attachment; filename=usuarios.pdf"
152       )
153       doc.pipe(res)
154       doc.fontSize(20).text("Reporte de Usuarios")
155       doc.moveDown()
156       datos.forEach((usuario)=>{
157         doc.text(`
158           ID: ${usuario.id}
159           Nombre: ${usuario.nombre}
160           Email: ${usuario.email}
161         `)
162       })
163     }
164   )
165 }
```

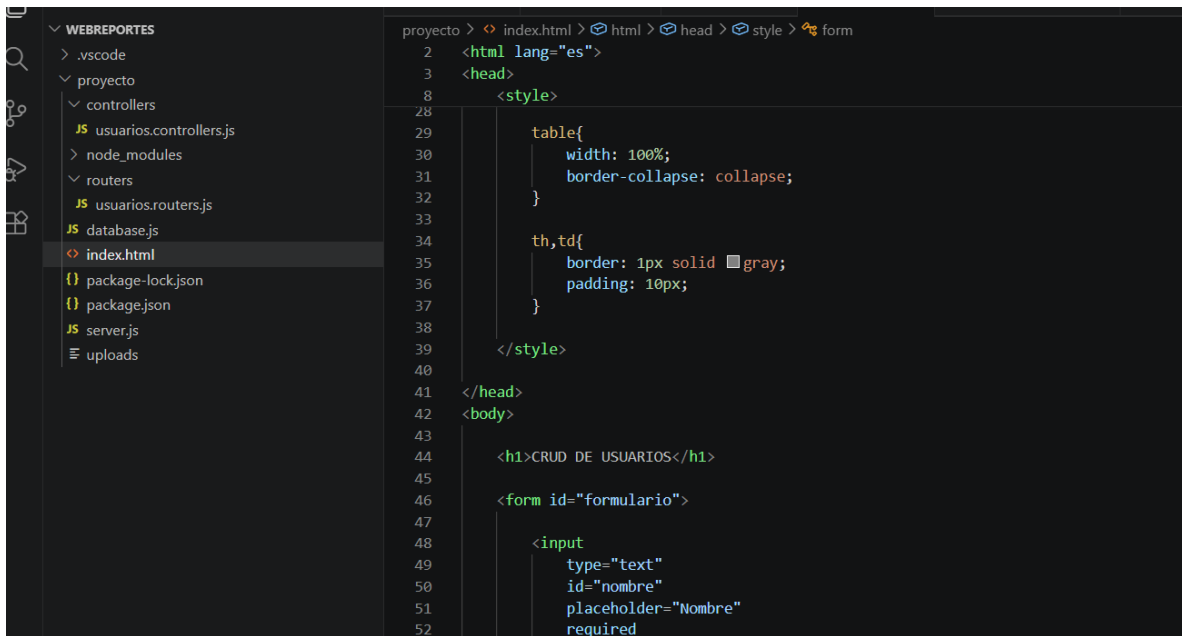


15. Corra el servidor, debe enviarle el mensaje servidor funcionando

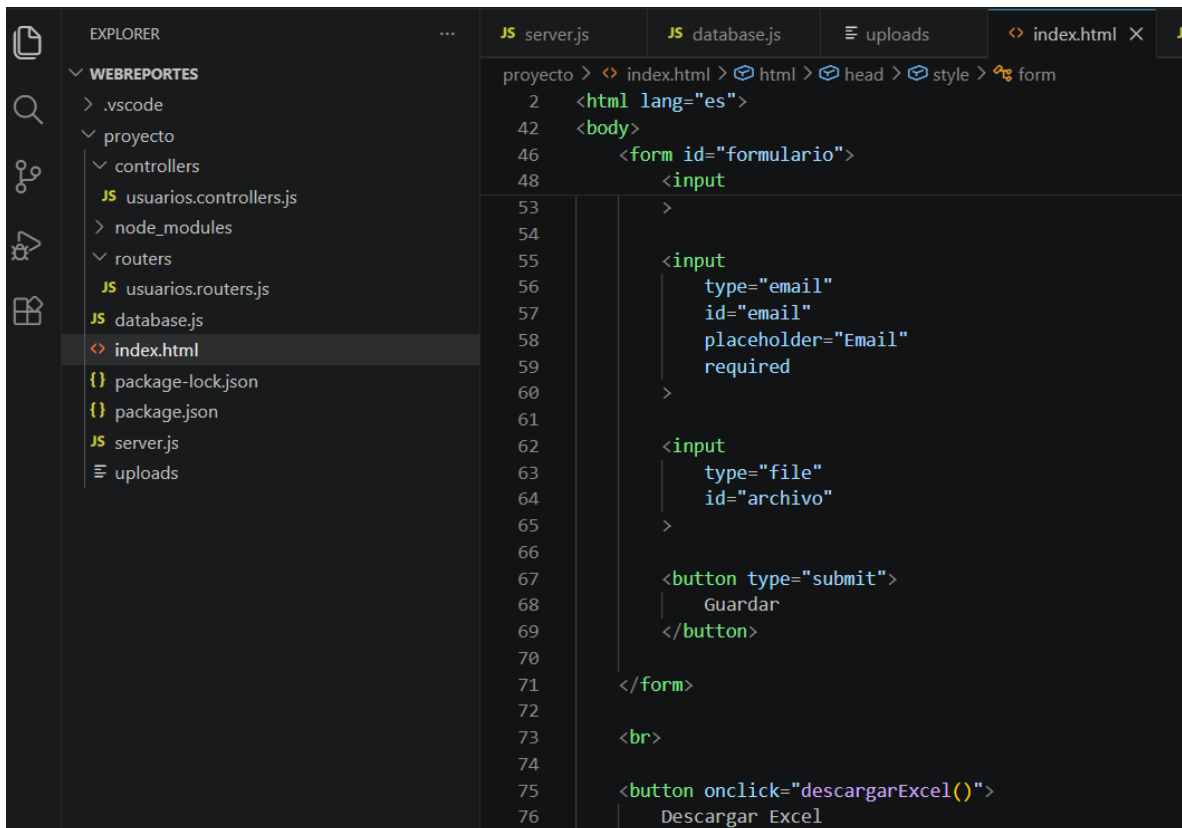
16. Por último para probar debe agregar el index.html para la vista del proyecto.

Agregue el siguiente código.

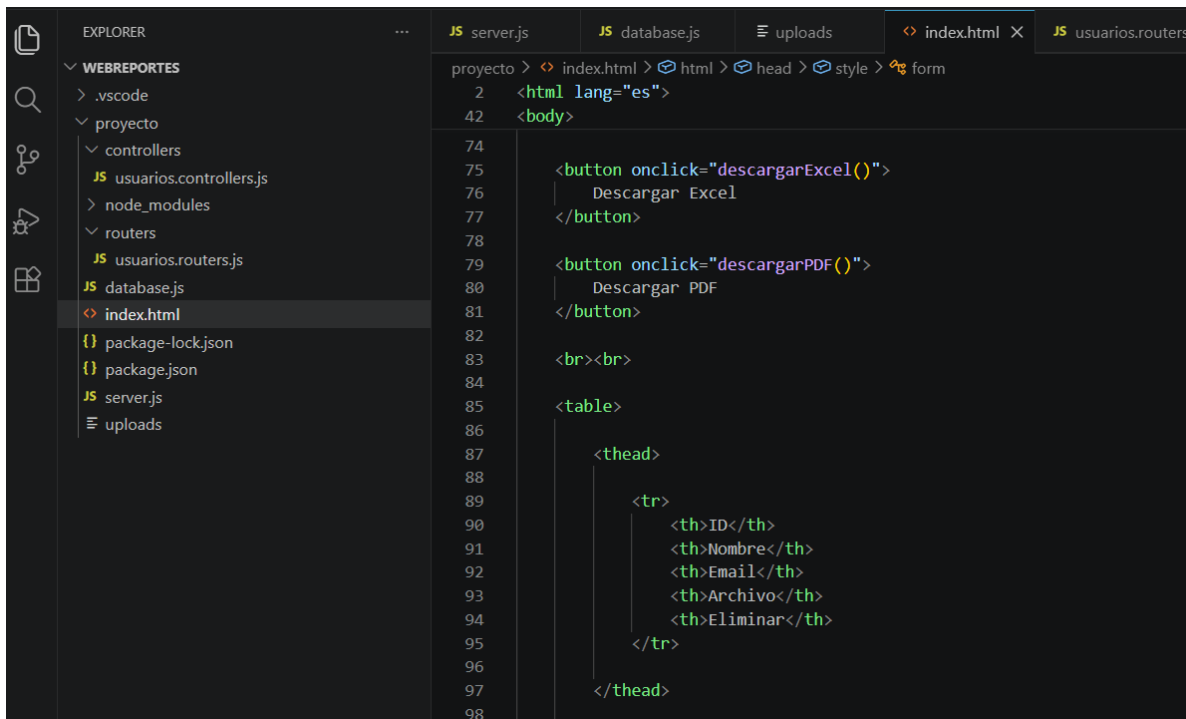




```
proyecto > index.html > html > head > style > form
2  <html lang="es">
3  <head>
8    <style>
28
29      table{
30        width: 100%;
31        border-collapse: collapse;
32      }
33
34      th,td{
35        border: 1px solid gray;
36        padding: 10px;
37      }
38
39    </style>
40
41  </head>
42  <body>
43
44    <h1>CRUD DE USUARIOS</h1>
45
46    <form id="formulario">
47
48      <input
49        type="text"
50        id="nombre"
51        placeholder="Nombre"
52        required
```

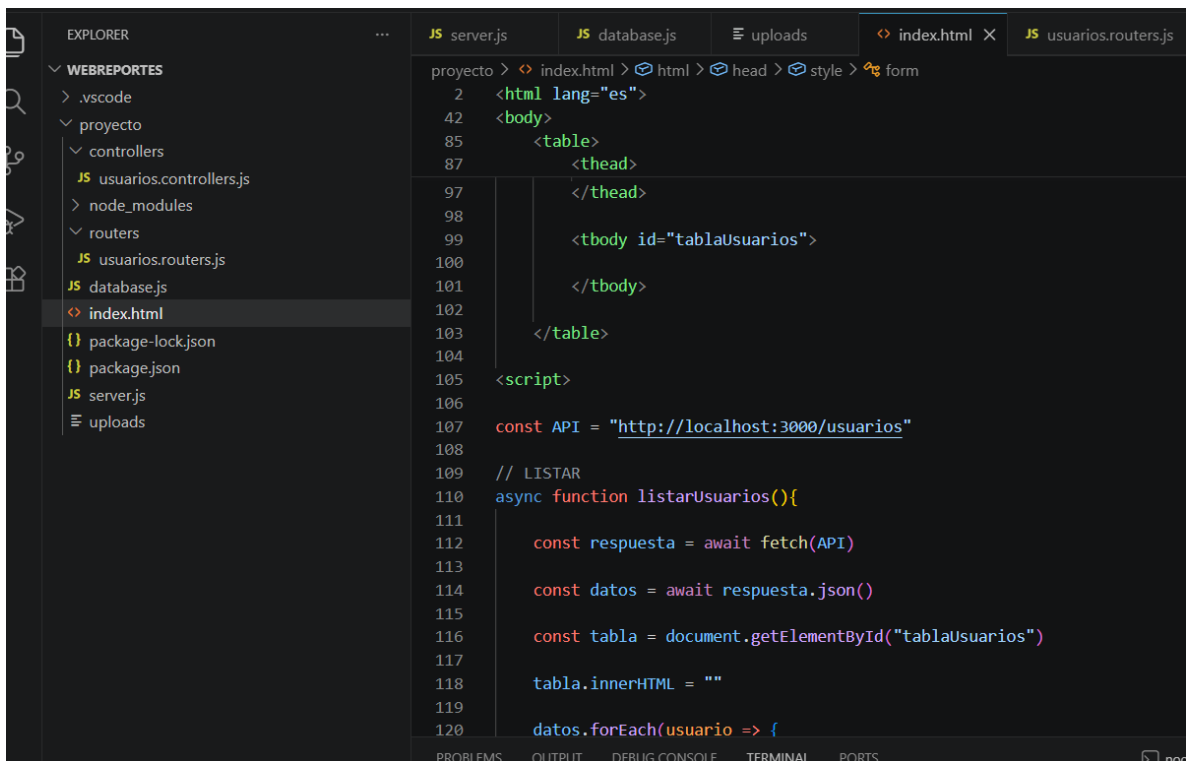


```
proyecto > index.html > html > head > style > form
2  <html lang="es">
42  <body>
46    <form id="formulario">
48      <input
53        >
54
55      <input
56        type="email"
57        id="email"
58        placeholder="Email"
59        required
60      >
61
62      <input
63        type="file"
64        id="archivo"
65      >
66
67      <button type="submit">
68        Guardar
69      </button>
70
71    </form>
72
73    <br>
74
75    <button onclick="descargarExcel()">
76      Descargar Excel
```

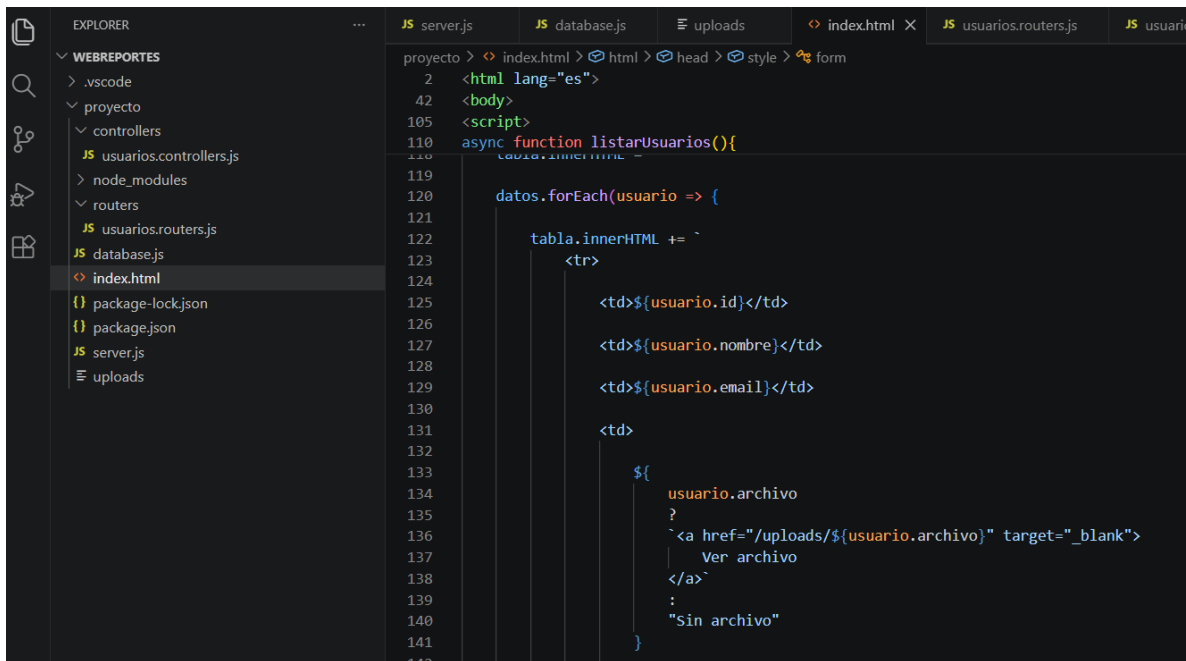


```
projecto > <> index.html > <> html > <> head > <> style > <> form
2  <html lang="es">
42 <body>

74
75   <button onclick="descargarExcel()">
76     Descargar Excel
77   </button>
78
79   <button onclick="descargarPDF()">
80     Descargar PDF
81   </button>
82
83   <br><br>
84
85   <table>
86
87     <thead>
88
89       <tr>
90         <th>ID</th>
91         <th>Nombre</th>
92         <th>Email</th>
93         <th>Archivo</th>
94         <th>Eliminar</th>
95       </tr>
96
97     </thead>
98
```



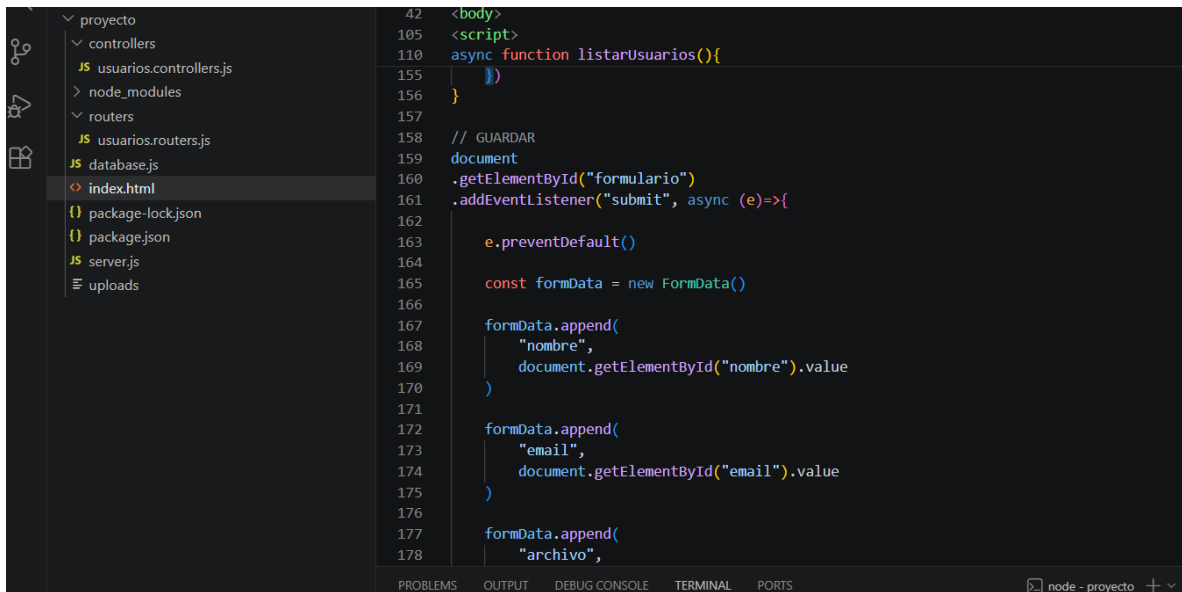
```
projecto > <> index.html > <> html > <> head > <> style > <> form
2  <html lang="es">
42 <body>
85   <table>
87     <thead>
97     </thead>
98
99     <tbody id="tablaUsuarios">
100
101     </tbody>
102   </table>
103
104
105   <script>
106
107     const API = "http://localhost:3000/usuarios"
108
109     // LISTAR
110     async function listarUsuarios(){
111
112       const respuesta = await fetch(API)
113
114       const datos = await respuesta.json()
115
116       const tabla = document.getElementById("tablaUsuarios")
117
118       tabla.innerHTML = ""
119
120       datos.forEach(usuario => {
121
```



```
proyecto > < index.html > < html > < head > < style > < form
2 <html lang="es">
42 <body>
105 <script>
110 async function listarUsuarios(){
111     datos.forEach(usuario => {
112         tabla.innerHTML += `
113             <tr>
114                 <td>${usuario.id}</td>
115                 <td>${usuario.nombre}</td>
116                 <td>${usuario.email}</td>
117                 <td>
118                     ${
119                         usuario.archivo
120                         ?
121                         `<a href="/uploads/${usuario.archivo}" target="_blank">
122                             Ver archivo
123                         </a>`
124                         :
125                         "Sin archivo"
126                     }
127                 </td>
128             </tr>
129         `
130     })
131 }
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
```

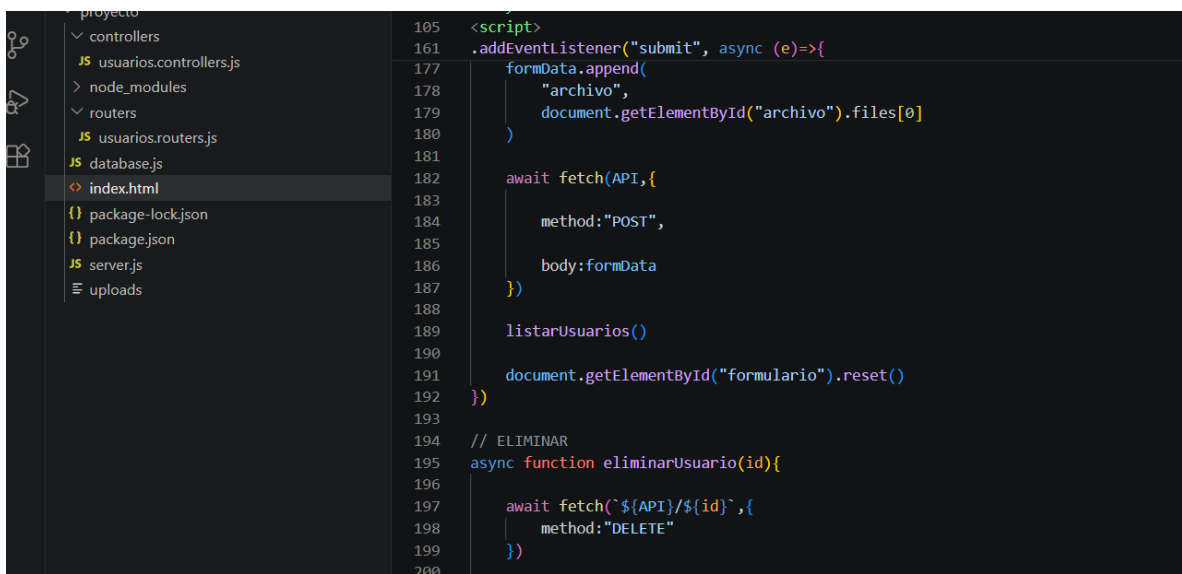


```
proyecto > < index.html > < html > < head > < style > < form
105 <script>
110 async function listarUsuarios(){
120     datos.forEach(usuario => {
137         Ver archivo
138         </a>`
139         :
140         "Sin archivo"
141     })
142
143     </td>
144
145     <td>
146         <button onclick="eliminarUsuario(${usuario.id})">
147             Eliminar
148         </button>
149     </td>
150
151     </tr>
152
153     </tr>
154
155     </tr>
156
157 }
```



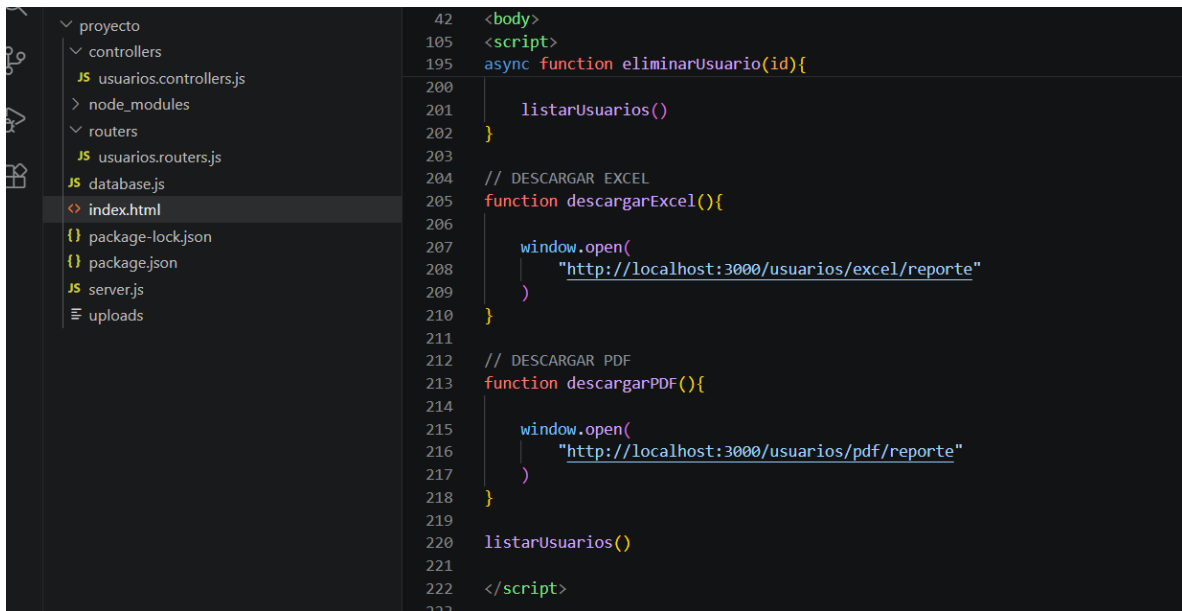
This screenshot shows the VS Code editor with the file explorer on the left and the code editor on the right. The file explorer shows a project structure with folders like 'proyecto', 'controllers', 'routers', and 'uploads'. The 'index.html' file is selected. The code editor displays the implementation of the 'listarUsuarios' function, which is an asynchronous function that prevents default form submission, creates a FormData object, appends form values, and sends a POST request to the API.

```
42 <body>
105 <script>
110 async function listarUsuarios(){
155 }
156 }
157
158 // GUARDAR
159 document
160 .getElementById("formulario")
161 .addEventListener("submit", async (e)=>{
162
163     e.preventDefault()
164
165     const formData = new FormData()
166
167     formData.append(
168         "nombre",
169         document.getElementById("nombre").value
170     )
171
172     formData.append(
173         "email",
174         document.getElementById("email").value
175     )
176
177     formData.append(
178         "archivo",
```

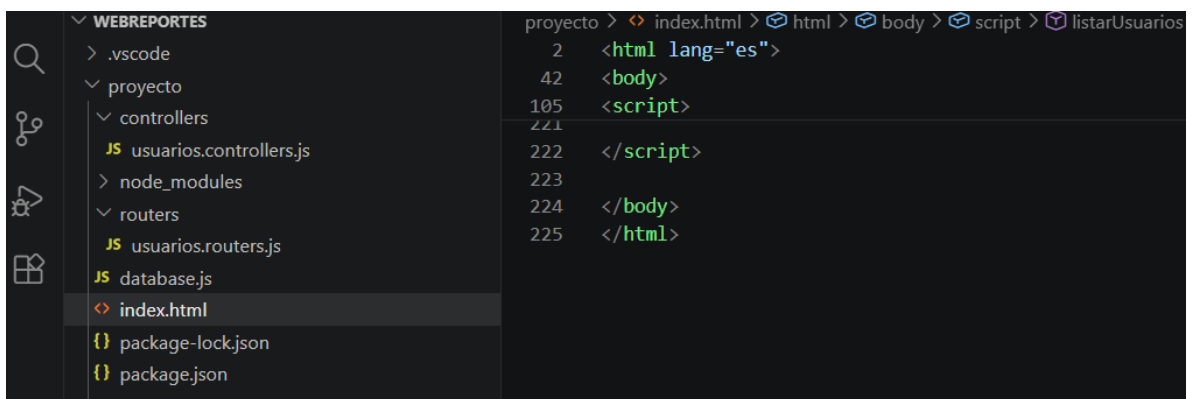


This screenshot shows the continuation of the 'listarUsuarios' function and the implementation of the 'eliminarUsuario' function in 'index.html'. The 'listarUsuarios' function continues by appending the 'archivo' file to the FormData object, sending a POST request, and resetting the form. The 'eliminarUsuario' function is an asynchronous function that sends a DELETE request to the API.

```
105 <script>
161 .addEventListener("submit", async (e)=>{
177     formData.append(
178         "archivo",
179         document.getElementById("archivo").files[0]
180     )
181
182     await fetch(API,{
183
184         method:"POST",
185         body:formData
186     })
187
188     listarUsuarios()
189
190     document.getElementById("formulario").reset()
191 })
192
193 // ELIMINAR
194 async function eliminarUsuario(id){
195
196     await fetch(`${API}/${id}`,{
197         method:"DELETE"
198     })
199 }
200
```



```
42 <body>
105 <script>
195 async function eliminarUsuario(id){
200
201     listarUsuarios()
202 }
203
204 // DESCARGAR EXCEL
205 function descargarExcel(){
206
207     window.open(
208         "http://localhost:3000/usuarios/excel/reporte"
209     )
210 }
211
212 // DESCARGAR PDF
213 function descargarPDF(){
214
215     window.open(
216         "http://localhost:3000/usuarios/pdf/reporte"
217     )
218 }
219
220 listarUsuarios()
221
222 </script>
223
```



```
proyecto > < index.html > html > body > script > listarUsuarios
2 <html lang="es">
42 <body>
105 <script>
221
222 </script>
223
224 </body>
225 </html>
```

Finalmente se despliega de la siguiente manera :

CRUD DE USUARIOS

Nombre

Email

Elegir archivo

No se ha seleccionado ningún archivo

Guardar

Descargar Excel

Descargar PDF

ID	Nombre	Email	Archivo	Eliminar
11	juan	jaun@gmail.com	Sin archivo	Eliminar
13	diego lopez	diego@gmail.com	Sin archivo	Eliminar
15	Alejandro	alejandro@gmail.com	Sin archivo	Eliminar
16	Flor Hernandez P	fmhernandez@unicauca.edu.co	Sin archivo	Eliminar
19	hugo	jijggg	Sin archivo	Eliminar
20	nuevo	nuevo@gmail.com	Sin archivo	Eliminar
21	nuevo abril	abril@gmail.com	Sin archivo	Eliminar

PDF

usuarios (1).pdf

Abrir archivo

PDF

usuarios.pdf

Abrir archivo

X

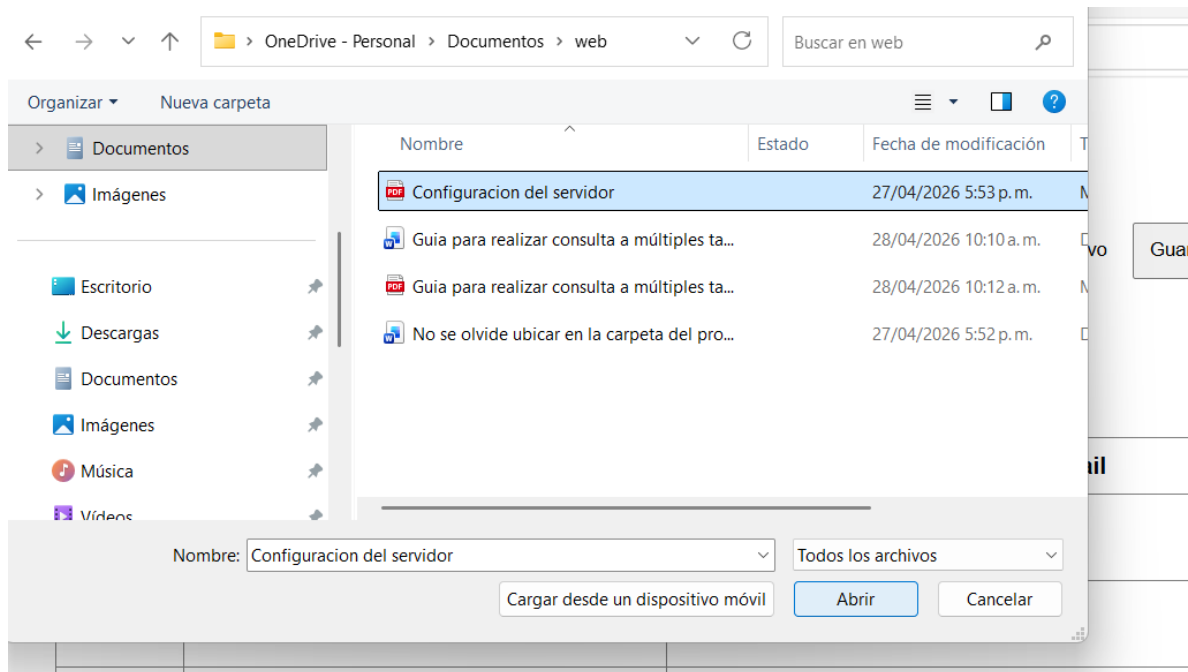
usuarios.xlsx

Abrir archivo

Ver más

usuarios (1).pdf

Funcionando las descargas, y carga de archivos



Reestructure el proyecto, poniendo los estilos en otro archivo. La entrega se realizara el próximo 18 de mayo el taller y en el proyecto 25 de mayo como entrega final.